

Week 3 - Day 13a Supplementary: Merge Sort

In this supplementary, we continue with our focus on sorting. Sorting is a fundamental concept in computer science that helps in organizing data in a particular order. We will see how to sort lists according to different sorting keys, as well as implement the famous merge sort algorithm without recursion.

Let's get started with the exercises!

Exercise 1: Sorting Heights [] []

You are given two arrays, `names` and `heights`, which are of both length n . For each index $0 \leq i \leq n-1$, `names[i]` and `heights[i]` denotes the name and height of the i -th person. So, for example, if

```
names = ["Mary", "John", "Emma"]
heights = [180, 165, 170]
```

then `Mary` is the tallest person, followed by `Emma` and then `John`. Your task is to return the names in **descending** order, meaning the name of the tallest person should be first, followed by the name of the second tallest person, and so on. If there are multiple people with the same height, then sort them alphabetically.

For example, if the input is

```
names = ["Tony", "Noam", "Abraham", "Georg"]
heights = [200, 160, 160, 170]
```

then the output should be `["Tony", "Georg", "Abraham", "Noam"]`. That is, because `Tony` is the tallest person, followed by `Georg`. `Abraham` and `Noam` have the same height, but `Noam` is alphabetically larger than `Abraham`, so you need to put `Noam` after `Abraham`.

```
# TODO: Your code here
def sort_heights(names, heights):
    pass

assert sort_heights([], []) == []
assert sort_heights(["Mary", "John", "Emma"], [180, 165, 170]) ==
["Mary", "Emma", "John"]
assert sort_heights(["Tony", "Noam", "Abraham", "Georg"], [200, 160,
160, 170]) == ["Tony", "Georg", "Abraham", "Noam"]

print("All tests passed. [ ]")
```

Exercise 2: Iterative Merge Sort [] [] []

In lecture and the exercises, you have seen how to implement merge sort recursively. However, it is also possible to implement it iteratively, i.e. using loops. Your function `mergesort_iter` should return the list, sorted in ascending order.

<summary><i>Hint</i></summary>

Maintain a nested list ``sublists`` containing sorted lists. In each pass of, pop the first two sublists, merge them together and add the merged list to the end of ``sublists``. In the end, once ``sublists`` contains only a single list, we extract and return the inner list.

```
# TODO: Your code here
def mergesort_iter(lst):
    sublists = [[i] for i in lst]
    pass

# Use this code cell to test your solution
assert mergesort_iter([]) == []
assert mergesort_iter([1]) == [1]
assert mergesort_iter([1, 2, 3]) == [1, 2, 3]
assert mergesort_iter([3, 2, 1]) == [1, 2, 3]
assert mergesort_iter(list(range(100, 0, -1))) == list(range(1, 101))

print("All tests passed. [ ]")
```

Exercise 3: Majority Elements [] []

Implement a function `majority` that given an array `nums` of size n , returns the majority element. The majority element is the element that appears more than $\lfloor n/2 \rfloor$ times. You may assume that the majority element always exists in the array, i.e. there always exists an element that appears more than $\lfloor n/2 \rfloor$ times. For example,

```
majority([3, 2, 3]) == 3
majority([2,2,1,1,1,2,2]) == 2
```

3.1 Implement the function using a dictionary. The time complexity of your solution should be $O(N)$, where N is the length of the list `nums`.

```
# TODO: Your code here
def majority(nums):
    pass

# Use this code cell to test your solution
assert majority([3, 2, 3]) == 3
assert majority([2, 2, 1, 1, 1, 2, 2]) == 2
```

```
assert majority([1] * 10 + [2] * 9) == 1
print("All tests passed. 🎉")
```

3.2 Now, try to implement the same function, but recursively. Do not use a dictionary.

```
# TODO: Your code here
def majority_rec(nums):
    pass

# Use this code cell to test your solution
assert majority_rec([3, 2, 3]) == 3
assert majority_rec([2, 2, 1, 1, 1, 2, 2]) == 2
assert majority_rec([1] * 10 + [2] * 9) == 1

print("All tests passed. 🎉")
```

Exercise 4: List Intersection [🔗]

Implement a function `intersection` that given two integer arrays `nums1` and `nums2`, return their intersection. The intersection of two arrays contains each element that appears in both arrays. If the element appears multiple times in both arrays, then it should appear in the result as many times as it shows in both arrays. For example,

```
intersection([1, 2, 2, 1], [2, 2, 2]) == [2, 2]
intersection([4, 9, 9, 5], [9, 4, 9, 8, 4]) == [4, 9, 9]
```

For your solution, **you are not allowed to use a dictionary**.

```
# TODO: Your code here
def intersection(nums1, nums2):
    pass

# Use this code cell to test your solution
assert sorted(intersection([], [])) == []
assert sorted(intersection([1, 2, 3], [])) == []
assert sorted(intersection([], [1, 2, 3])) == []
assert sorted(intersection([1, 2, 2, 1], [2, 2])) == [2, 2]
assert sorted(intersection([4, 9, 9, 5], [9, 4, 9, 8, 4])) == [4, 9, 9]

print("All tests passed. 🎉")
```

Congratulations 🎉